



2



7



840 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

تطبيق العمليات المنطقية

على prolog

عملي مشترك



RB Informatcs; 4/5/2025

مبادئ الذكاء الصناعي

السلام عليكم يا أصدقاء

تعرفنا في محاضرتنا السابقة على أساسيات لغة الـ prolog وقمنا بتطبيق بعض الأمثلة سوياً، وفي هذه المحاضرة سنقوم بتطبيق المزيد من الأمثلة والتمارين ذات الأفكار الجديدة والمهمة. بسم الله نبدأ.

تمرين :

لدينا أربع أنواع من الأطعمة:

سلطة salad - بيتزا pizza - لحم meat - سمك fish

ولدينا هؤلاء الأشخاص مع تفضيلات الطعام الخاصة بهم:

- Ahmad likes meat and salad
- Lama likes salad and pizza
- Samer likes pizza and meat
- Ali likes fish
- Omar likes fish.

ولدينا القواعد التالية:

- If Lama likes a food, then Sarah also likes it.
- Two people have the same taste if they both like the same food.
- A person is vegetarian if they like salad or pizza and do not like meat.

نريد تمثيل هذه المعلومات باستخدام prolog وأن نستطيع الاستعلام عن ما يلي:

- What food does Sarah like?
- Who is vegetarian?
- Who has the same taste as whom?



▪ **نبدأ** أولاً مع تمثيل الأطعمة كما تعلمنا في المحاضرة السابقة:

food(salad).
food(pizza).
food(meat).
food(fish).

▪ **ثم** نقوم بتمثيل الأشخاص وتفضيلاتهم أيضاً بنفس الطريقة:

likes(ahmad, meat).
likes(ahmad, salad).
likes(lama, salad).
likes(lama, pizza).
likes(samer, pizza).
likes(samer, meat).
likes(ali, fish).
likes(omar, fish).

▪ **أما** بالنسبة لتمثيل القواعد، سنبدأ بتمثيلهم معاً خطوة بخطوة:

🔗 القاعدة الأولى:

If Lama likes a food, then Sarah also likes it.

سنقوم بتمثيلها كما تعلمنا في المحاضرة السابقة:

likes(sarah, Food) :- likes(lama, Food).

عند تطبيق ال query للاستعلام عن الطعام الذي تحبه سارة:

?- likes(sarah,X).

نجد أن الخرج:

X = salad ;

X = pizza.



في زحام و ضجيج الحياة لا تنس الغاية التي
خلقت من أجلها لك شيء في هذا العالم فقم

القاعدة الثانية:

Two people have the same taste if they both like the same food.

لتمثيل هذه القاعدة، يجب أن يتحقق أن الشخصين Person1, Person2 لهما نفس ذوق الطعام
 same_taste(Person1, Person2) ، وكي يتحقق أنه لهما نفس ذوق الطعام ، يجب أن يتحقق أنه يوجد نوع طعام
 مشترك بينهما Food، وكي نستطيع التحقق من هذا يجب أن يتحقق الشرطين التاليين معاً:

الشرط الأول:	الشرط الثاني:
أن يحب الشخص الأول Person1 أحد أنواع الطعام Food.	أن يحب الشخص الثاني Person2 نفس نوع الطعام الذي يحبه الأول Food.
likes(Person1, Food).	likes(Person2, Food).

وإذا تحقق هذان الشرطان معاً، سينتج أن Person1 و Person2 لهما نفس ذوق الطعام، ولنستطيع تمثيل
 هذه القاعدة كاملةً يجب أن نربط بين الشرطين السابقين باستخدام العملية AND.

فائدة (1):

كيف نستخدم العملية AND للربط بين الحقائق ؟ نضع فاصلة (,) بين الجمل التي نريد الربط بينها.

الشكل النهائي للقاعدة:

same_taste(Person1, Person2) :-

likes(Person1, Food),

likes(Person2, Food).

أي أنه إذا أحب الشخص Person1 الطعام Food وإذا أحب الشخص Person2 الطعام Food نفسه، فإن
 Person1 و Person2 لهما ذوق الطعام نفسه.

ولكن في حال كنا نريد معرفة من هم الأشخاص أصحاب الذوق نفسه، وقمنا بتطبيق هذا الـ query:
 ?- same_taste(X, Y).

أي أننا نريد معرفة جميع الأشخاص X و Y المحققة للشروط، سيظهر لدينا خرج طويل وفيه تكرارات
 بشكل كبير (يمكنكم تجربة تطبيق الـ query على القاعدة بشكلها الحالي وتتبع الخرج الناتج) والسبب
 في هذا التكرار هو أن الـ prolog تقوم بتجريب جميع الأشخاص مرة في X ومرة في Y.
 فمثلاً أول مشكلة سنلاحظها في الخرج هي وجود:

X = Y, Y = ahmad وهذا يقابل أن ahmad و ahmad لهما نفس الذوق ولكنهما شخص واحد
 في الحقيقة فالنتيجة غير منطقية.

ولحل هذه المشكلة سنضيف شرط ثالث وهو ألا يكون Person1 مساوياً لـ Person2.

فائدة 2:

كيف نكتب "لا يساوي" في Prolog ؟ نضع (=) بين المتغيرين المطلوبين.

■ فتصبح القاعدة:

```
same_taste(Person1, Person2) :-
likes(Person1, Food),
likes(Person2, Food),
Person1 \= Person2.
```

■ وهكذا سنتخلص من النتيجة الموجودة سابقاً، ولكن ستبقى مشكلة التكرارات موجودة في الخرج، فمثلاً ahmad و lama لهما نفس الذوق **لأن** كلاهما يحبان الـ salad، ولكن عند تطبيق الـ query سنلاحظ أنه سيظهر في الخرج مرة $X = ahmad, Y = lama$ ومرة $X = lama, Y = ahmad$ رغم أنهما نتيجة واحدة، ولحل هذه المشكلة يجب أن نلغي التكرارات بحيث لا يجرب جميع التباديل الممكنة.

فائدة 3:

كيف نستطيع إلغاء التكرارات في العلاقات بين عنصرين؟ نستخدم التعليمة (<) بين المتغيرين المطلوبين.

■ فتصبح القاعدة:

```
same_taste(Person1, Person2) :-
likes(Person1, Food),
likes(Person2, Food),
Person1 @< Person2.
```

ماذا يعني الرمز (<) وكيف يتخلص من التكرارات؟

عند كتابة هذا الرمز بين متغيرين يقوم بمقارنتهما حسب الترتيب الأبجدي (أو ترتيبها المعجمي مثلاً abb أصغر من baa)، وبالتالي سيكون الشرط أن اسم الـ Person1 يجب أن يكون أصغر تماماً (أبجدياً) من اسم الـ Person2، وبهذه الطريقة نتخلص من التكرار بشكل كامل ويكون خرج الـ query هو:

```
X = ahmad,
Y = samer ;
X = ahmad,
Y = lama ;
X = ahmad,
Y = sarah ;
X = lama,
Y = sarah ;
X = lama,
Y = samer ;
```

X = lama
 Y = sarah ;
 X = samer,
 Y = sarah ;
 X = ali,
 Y = omar ;
 false.

ملاحظات عن الخرج:

- تكررت X = lama, Y = sarah مرتان بسبب قاعدة أن sarah تحب نفس الطعام الذي تحبه lama.
 - يوجد false في النهاية لأن prolog لا يستطيع تنفيذ المقارنة $Y @< X$ إذا كانت المتغيرات X و Y غير مرتبطة بقيم محددة وقت التقييم، وهي تابعة لآلية عمل prolog من الداخل.

كيف سيكون شكل القاعدة لو كان الطلب يشترط أن يتشارك الشخصان بحبهما لنفس أنواع الطعام تماماً دون أن يكفي صنف واحد فقط من الأطعمة ليكون لهما الذوق نفسه؟

يجب أن نقارن قائمة الطعام الذي يحبه الشخص الأول مع قائمة الطعام الذي يحبه الشخص الثاني، وفي حال تطابقت بشكل كامل يتحقق شرط أن لهما الذوق نفسه.

فائدة 4:

يمكننا تجميع النتائج التي تحقق شرط معين في قائمة مرتبة وبدون تكرار باستخدام setof/3 وصيغتها كما يلي:

setof(X, Goal, Set).

وتعني إبحث عن جميع القيم الممكنة لـ X التي تحقق الشرط Goal وضعها في القائمة Set (وهي قائمة مرتبة وبدون تكرار).

■ فتصبح القاعدة:

same_taste(Person1, Person2) :-
 setof(F, likes(Person1, F), Foods1),
 setof(F, likes(Person2, F), Foods2),
 Foods1 == Foods2,
 Person1 @< Person2.

■ أي قم بتخزين جميع الأطعمة التي يحبها الشخص Person1 في القائمة Foods1 وقم بتخزين جميع الأطعمة التي يحبها الشخص Person2 في القائمة Foods2، وفي حال تطابقت القائمتان تماماً (دون زيادة أو نقصان) سيكون للشخصين نفس الذوق بالطعام.

ملاحظة: يمكن الاستغناء عن الشرط $Foods1 == Foods2$ وتسمية القائمتين بنفس الاسم $Foods$ مثلاً ، وسيعني هذه ضمناً أن القائمتين يجب أن تكونا متطابقتين

القاعدة الثالثة:

A person is vegetarian if they like salad or pizza and do not like meat.

في هذه القاعدة، يكون الشخص P نباتي، إذا كان يحب السلطة أو البيتزا، وكان لا يحب اللحم. نستطيع تمثيل هذه القاعدة بنفس طريقة القاعدة السابقة ولكن سنحتاج لإضافة عملية الـ OR، وعملية النفي NOT.

فائدة 5:

كيف نستخدم العملية OR للربط بين الحقائق؟ نضع فاصلة منقوطة (:) بين الجمل التي نريد الربط بينها.

فائدة 6:

كيف نستخدم العملية NOT لنفي الحقائق؟ نضع (+\) قبل الحقيقة التي نريد نفيها.

الشكل النهائي للقاعدة:

vegetarian(Person) :-
(likes(Person, salad); likes(Person, pizza)),
\+ likes(Person, meat).

لماذا قمنا باستخدام الأقواس بين الحقائق وماذا سيحصل لو لم نقم باستخدامها؟

- في prolog وفي معظم لغات البرمجة، يتم قراءة وتطبيق الجملة من اليمين إلى اليسار، ولذلك في حال لم نستخدم الأقواس، سيتم اعتبار أن الشخص نباتي في حال تحقق أنه:
(لا يحب اللحم ويحب البيتزا) أو (يحب السلطة)
وهذه العبارة خاطئة حيث أن العبارة الصحيحة هي:
أن يحب (البيتزا أو السلطة) و (لا يحب اللحم)
- وبالتالي نضع الأقواس ليتم تطبيق القاعدة بالشكل الصحيح.

عند تطبيق الـ query للاستعلام عن الأشخاص النباتية:

?- vegetarian(X).

نجد أن الخرج:

X = lama ;
X = sarah ;
X = lama ;
X = sarah.



ولكن لماذا يوجد تكرار؟

لأنه يقوم أول مرة بمقارنة شرط أن يحب الشخص البيتزا مع شرط ألا يحب اللحم، ويقوم ثاني مرة بمقارنة أن يحب الشخص السلطة مع شرط ألا يحب اللحم، ولذلك نجد تكرار في الخرج.

فائدة 7:

من أجل وضع comment على سطر، نستخدم % قبل هذا السطر.

أما من أجل وضع comment لمجموعة أسطر، نضع مجموعة الأسطر بين: /* */

طلب إضافي على التمرين السابق :

كيف يمكننا التخلص من التكرار الحاصل في خرج الـ query السابق (query الـ vegetarian(X))؟

سنضع حل هذا الطلب في المحاضرة القادمة بإذن الله ☺

<The End>